

VS Code 的 QA 缺陷提交 Agent

BugIt

把粗略的测试笔记变成精炼、经过审阅的缺陷工单——几秒钟内提交到你的 tracker。

一个能学习你团队工作流程的 QA agent。

由一位每天撰写和审阅缺陷报告的资深 QA 工程师打造。

🔒 设计上私密——你的工作内容绝不会发送给 Taskivator。

~10 分钟

每条缺陷节省的时间

重复检查

提交前进行

~15 分钟

一次性设置

\$39.99/年

早鸟价 · 之后 \$59.99

快速上手指南 · 阅读一次，约 15 分钟完成设置，随后提交规范工单。

从这里开始

60 秒术语表

无需技术背景——只需了解这几个关键词，全用大白话讲清。

VS Code 微软出品的免费应用——这个 agent 所在的"窗口"。	GitHub Copilot 负责撰写报告的 AI 大脑。总成本为 BugIt 年度许可证，外加 Copilot 套餐或你自己的 AI 提供商用量费用。
BugIt 你在这个窗口中的助手。像给同事发消息一样聊天。	Tracker 你团队存放缺陷的地方（Jira、ADO、GitHub.....）。
MCP server 连接到你 tracker 的小"桥梁"。打开一次即可。	Token 桥梁使用的私密密码。安全保存。
config.json 一张设置表。agent 会替你填好。	Chat 你输入内容的地方："Write a bug report: app crashes on save."

只要你会安装应用、登录、打字造句，你就能用好它。

费用与授权

它的费用

搭配你喜爱的 AI 模型使用。BugIt 调度 GitHub Copilot 内部的模型（GPT、Claude 或 Gemini）——也可以用你自己的密钥独立运行。

免费 Copilot 免费版可满足轻度使用。	约 \$10/月 Copilot Pro，适合每天使用。
无需 API 密钥 无需托管，无按缺陷计费。	没有 Copilot ？ 自带你自己的 OpenAI/Anthropic 密钥。

\$39.99/年

早鸟价，之后 \$59.99/年——按年计费，同时 1 台设备，包含所有功能。团队版：5 个席位早鸟价 \$199/年，之后 \$249.99/年。

这还不到**每月 \$3.50**——比两个月的 ChatGPT Plus 还便宜，却能用一整年。同类 QA 工具是**按月**收费的；省下几个缺陷的处理时间，就已经值回票价。

团队许可证

一条命令即可把你的 tracker 设置、术语表和缺陷风格分享给全部 5 个席位—— `share.py export` ，队友 `import` ——让每个人从第一天起就用完全相同的格式提交。

物有所值

你实际获得的东西

不是"会写缺陷的 AI"——而是一个契合你团队现有工作方式的 QA agent。

没有 BugIt	有了 BugIt
手动撰写每一个缺陷	一句话 → 一份完整、经过审阅的工单
重复项在看板上不断堆积	提交前自动查重，即使措辞不同也能发现
每个测试人员的格式都不一样	每次都是同一套一致的模板
手动翻译并查找规格文档	多语言支持，并替你提取上下文

支持你团队已在使用的 tracker

BugIt 会为任意 tracker 撰写工单，并通过你启用的连接器提交——随时可切换。

引导式设置，经过测试的字段映射——自动设置严重程度/优先级：

- Jira Cloud
- Azure DevOps

引导式知识库来源（Atlassian Roovo MCP）：

- Confluence Cloud

实验性——由 BugIt 设置并在你的账户上实时验证的已知端点：

- Sentry
- GitHub
- Linear
- Notion

仅提供设置指导——通过你自己的兼容 MCP server：

- GitLab
- Shortcut
- YouTrack
- Bugzilla
- ClickUp
- Asana
- Trello
- Crashlytics
- BugSnag
- TestRail
- Xray
- Slack
- Teams

自动化设置不支持：

- S3
- Google Drive
- Azure Blob

此外，它还会检查重复项、查询崩溃信息、提取规格文档上下文、跨语言翻译，并撰写验证/关闭评论。

设置

6 步即可上手

在 VS Code 内完成一次性设置。agent 会安装任何缺失的组件。

- 1

安装 VS Code

从 code.visualstudio.com 下载并打开。
- 2

安装 GitHub Copilot

Extensions → 搜索 "GitHub Copilot" → Install → 登录。
- 3

打开此文件夹

File → Open Folder。agent 从 .github/agents/ 中加载。
- 4

选择 agent

在聊天中，把模式下拉菜单切换到 **BugIt QA Agent**。

5 输入 "Begin setup"

回答几个问题；agent 会写好 `config.json`。如果中途被打断，再说一遍就会从中断处继续，而不是重新开始。

6 启动 MCP server，说声 "hi"

打开你的 bridge——你就可以提交了。默认情况下，你的许可证密钥通过掩码终端提示输入，绝不会出现在聊天记录中。

快捷方式：`preset.py game` 从模板开始 · 没有 Copilot 就用 `setup_wizard.py` · `doctor.py` 获取就绪检查清单。

附带完整的分步操作手册。

购买后，你会获得一份详细的、逐步点击讲解的设置与用户指南——中文版——专为从未做过类似操作的人编写，让设置变得轻松，绝不会卡住。

购买前——你需要准备什么

需求	原因
VS Code（免费）	agent 在其中运行
GitHub Copilot（推荐）	为 agent 提供动力——或自带你自己的 OpenAI/Anthropic 密钥
Python 3.10–3.13	运行辅助工具
一个 tracker 账户	缺陷的去处——Jira 和 Azure DevOps 提供引导式设置和经过测试的字段映射（Azure DevOps 通过 Microsoft 的远程 MCP 公开预览接入）。GitHub 和 Linear 为实验性，并会在你的账户上实时验证；其他所有 tracker 都通过你自己的兼容 MCP server 连接

需要了解的事

BugIt 运行在 **VS Code** 中（而非其他聊天应用）；它调度**你自己的** AI 模型，而非自带模型；经过测试的字段映射（自动设置 severity/priority）为 **Jira + Azure DevOps**（Azure DevOps 通过 Microsoft 的远程 MCP 公开预览接入），Confluence 使用同一套引导式 Atlassian 连接，Sentry、GitHub、Linear 和 Notion 为实验性并会在你的账户上实时验证，其余一切都通过你自己的兼容 MCP server 连接；脱敏等其他保障措施是辅助手段，而非保证——**每份工单都由你审阅并批准**，并且在更强的 AI 模型上，效果会比非常轻量/免费的模型更加一致。

看看实际效果

输入一句话，得到一份规范工单。

这就是它的全部意义——像跟同事说话一样输入它；剩下的交给 agent。

YOU TYPE

"the app crashes every time I tap Save on iPhone"



BUGIT 起草工单——你审阅，然后输入 FILE

[Crash] App closes when tapping Save (iPhone)

步骤	1. 打开应用 2. 做任意修改 3. 点击保存
预期	修改会被保存，应用保持打开
实际	应用立刻关闭——每次都是
详情	重现率 5/5 · 严重程度 Critical · 平台 iOS (iPhone)

然后它会查重并提交到你的 tracker（在你输入 FILE 之后）——把 tracker 自己的优先级字段设置为匹配值（而非通用默认值）——并把链接交给你。想先调整一下？直接说 "make severity high"。想练习？说 dry run，它会写出整份内容而不提交。

日常使用，你只需输入

你输入	你得到
Write a bug report: ...	完整报告、查重、版本自动识别
Close #ID · Translate to ja: ...	验证/关闭评论 · 术语表统一的翻译
Update · Back up my settings · help	签名更新 · 本地快照 · 完整命令列表

可选附加功能

强力功能详解

在 Begin setup 中开启这些功能，然后从聊天中触发它们。

分类摘要

输入 Triage digest 获取即时站会摘要——提交/变更了什么、按严重程度和类别的数量统计。

导出缺陷到文件

Export bug to file 会把报告保存为本地 Markdown/JSON 而不提交——非常适合离线使用或留存副本。

撤销上次提交

Undo last filing 会调出 agent 最近提交的工单或评论，经你确认后，在你 tracker 允许的范围内将其删除或关闭。

SLA 标记

高严重程度的缺陷在提交时会被打上 SLA/截止标记，紧急问题会自动凸显。

分类专属字段

每个类别都会添加自己的字段（崩溃 → 堆栈跟踪，UI → 截图），确保每份报告都记录到恰当的细节。

全部可选。跳过它们，核心的撰写→审阅→提交流程依然完美运作。

值得信赖

内置的安全与保障

这些不是附加项——它们默认开启，也是团队愿意用它提交真实工单的原因。

🔒 我们绝不会看到你的工作内容

BugIt 运行在你的编辑器中；你的工单只会发送给你连接的 AI 模型和 tracker。唯一会到达我们这边的，是许可证/更新数据——你的密钥、一个单向哈希的设备 ID，以及你在首次激活时确认的席位标签（必填；可用姓名、用户名或邮箱，用来区分设备，且不作验证）。没有遥测，没有分析，绝不会有。

💾 自动备份

每次更新前——以及任何你需要的时候——你的配置、术语表、团队风格、MCP 端点和学习数据都会在你的设备上生成快照（许可证密钥和凭据值不包含在内）。一条命令即可恢复：`Restore my settings`。

✅ 未经你同意，绝不提交

每条工单和评论都会预览；不可逆的提交需要你输入 `FILE`（创建工单）或 `COMMENT`（发布评论）；仅输入“是”不会执行。说 `dry run` 即可进入零写入的练习模式——干运行会阻止 BugIt 内置的 Python 助手写入，并指示代理拒绝对跟踪器的写入；该拒绝是遵循 BugIt 指令，并非平台级锁定；评估时请使用只读凭据。

🔑 经过签名验证的更新

更新经过加密签名，签名密钥与授权体系分开保管。agent 会逐字节校验，拒绝任何被篡改的内容——然后保留你所有的设置。

🔑 磁盘上不含任何密钥

令牌保存在你操作系统的凭据库中，绝不写入文件。`config.json` 只保存组织和 URL——而且验证器会在任何内容被保存前（无论是在配置中，还是在 agent 为你运行的命令中）标记出看起来像泄露密钥的内容。

🛡️ 抵御提示注入

从页面或工单中提取的文字会被当作数据处理，绝不当作命令——被注入的指令会被标记，而非执行，并会呈报给你。

🔧 安全自定义

`config.json`、团队风格、术语表和 MCP 设置都属于你——每次更新都会保留。agent 文件和其他 `.github/instructions/` 文件则不是：手动编辑它们会被静默覆盖，且没有备份可以恢复。

设计上私密。你的缺陷报告、规格文档、术语表和设置始终属于你。agent 只与你连接的工具和你自己的 AI 模型通信——绝不会发给我们。

尊重你的授权机制：成功完成在线激活后，缓存的许可证可让此设备离线工作最多 7 天，之后需要在线验证。同时只能在一台设备上使用；可自由更换设备。

支持

故障排查

症状	解决办法
"No tracker enabled"	运行 <code>Begin setup</code> ，选择一个 tracker
Server 显示 ❌	打开 <code>.vscode/mcp.json</code> ，点击 Start；或说 "fix servers"
要求重新登录	输入 "fix servers"，重启服务器并在浏览器中完成登录；BugIt 从不显示已保存的凭据值
无法提交	检查 <code>validate_config.py --test</code> ；确认主 tracker 已连接且已登录
看起来偏向游戏	把 <code>config.categories</code> 编辑成你的领域

还是卡住？访问 bugit.dev 或发邮件至 support@bugit.dev——真人提供帮助（技术支持仅提供英文）。购买后 7 天内遇到设置问题？我们会帮你把它跑起来，或协助你通过商店办理退款。

一整年的费用，不到两个月 ChatGPT Plus 的价钱

\$39.99/年不到**每月 \$3.50**，激活当天起才开始计时——没有按缺陷收费。同类 QA 工具是**按月**收费的。省下几个缺陷的处理时间，就已经值回票价。

每条缺陷节省约 10 分钟

一句话 → 完整工单

支持你的 tracker

你现在用的那一个

重复检查

提交前进行

多语言

10+ 种语言，术语表锁定

我打造 **BugIt**，是因为多年来每天撰写和审阅缺陷报告之后，我厌倦了把时间花在重复的排版上，而不是发现真正的问题。**BugIt** 就是我一直希望拥有的那个工具。

语言支持：en, ja, fr, de, es, pt-br, it, ko, zh, ru 及更多 · Solo 版 \$39.99/年 早鸟价（之后 \$59.99）· Team 版 5 席位 \$199/年 早鸟价（之后 \$249.99）

BugIt · 许可证有效期内可获得同一大版本内的免费更新 · 一次购买 = 一位用户（不可转售或再分发）

© 2026 Taskivator. All Rights Reserved. · bugit.dev